Become a Git Power User

The commands and settings from the free course at consciouslycode.com/training. Lesson numbers in grey.

Set up once 2, 3

```
git config --global
init.defaultBranch main
git config --global pull.ff only
git config --global
push.autoSetupRemote true
git config --global fetch.prune true
git config --global
merge.conflictStyle zdiff3
git config --global rerere.enabled
true
git config --global rebase.autoSquash
true
git config --list --show-origin --
show-scope
```

```
[includeIf "gitdir:~/git/work/"]
  path = ~/.gitconfig-work
```

Work email in work repos, automatically.

What git stores 1

`git cat-file -p HEAD`
A commit: tree, parents, author, message.

`git cat-file -p 'HEAD^{tree}'`
A folder listing: blobs and trees.

`cat .git/HEAD`
Which branch you're on. A branch is a file holding one id.

Commit one thing at a time 7

`git add -p`
Stage hunk by hunk: y, n, s (split), e (edit).

`git diff --staged`
What the next commit will contain.

`git commit --amend --no-edit`
Add staged changes to the last commit.

Undo anything 8

`git restore <file>`
Discard uncommitted edits. Can't be undone.

`git restore --staged <file>`
Unstage, keep the edit.

`git reset --soft HEAD~1`
Take back the last commit, keep it staged.

`git reflog`
Every commit HEAD has been on.

`git reset --hard HEAD@{1}`
Back to where you were one step ago.

`git reset --hard ORIG_HEAD`
Undo the last merge, rebase, or reset.

`git branch <name> <id>`
Bring back a deleted branch (branch -D printed the id).

`git revert <commit>`
Already pushed? Add a commit that undoes it.

Branches and worktrees 9, 10

`git switch -c feature/x`
New branch from where you are.

`git push -u origin feature/x`
Share it and track it.

`git push --force-with-lease`
After a rebase; refuses if someone else pushed.

`git branch -d x`
Delete if merged. -D deletes anyway.

`git worktree add ../hotfix -b fix/y main`
A second checkout, no stash needed.

`git worktree list / remove`
See and clean up worktrees.

Remotes and forks 11

```
git remote add upstream <url>
git fetch upstream
git merge --ff-only upstream/main
git push origin main
```

Clean up before review 12

`git commit --fixup=<commit>`
A fix for an earlier commit.

`git rebase -i --autosquash main`
Fold fixups in; edit the todo list for the rest.

`git diff branch@{1} branch`
No output: the code didn't change.

`git range-diff main branch@{1} branch`
What changed between versions, per commit.

`git rebase --update-refs main`
Keep stacked branches together.

Landing a branch 13

`git merge --no-ff feature`
Keep the branch's commits and a merge commit.

`git rebase main && git merge --ff-only`
Linear history, commits kept.

`git merge --squash feature`
One commit on main.

Find the commit that broke it 15

```
git bisect start HEAD v1.0
git bisect run ./test.sh   # 0 good,
1 bad, 125 skip
git bisect reset
```

`git log -S'text'`
Commits that added or removed a string.

`git log -G'regex'`
Commits whose diff touches a matching line.

`git log -L :func:file`
History of one function.

`git blame -w / blame.ignoreRevsFile`
See past whitespace and reformat commits.

Hooks 14

`git config core.hooksPath .githooks`
Hooks committed with the repo. Make them executable.

`git commit --no-verify`
Skip pre-commit and commit-msg, once.

Aliases 16

```
[alias]
  st = status -sb
  lg = log --graph --
pretty=format:'%C(auto)%h%d %s
%C(dim)(%cr, %an)%C(reset)'
  unstage = restore --staged
  amend = commit --amend --no-edit
  pushf = push --force-with-lease
  last = log -1 --stat
```

Tags and signing B1

`git tag -a v1.0 -m '...'`
Annotated tag for a release.

`git describe --dirty`
v1.0-2-g0c27521: a version for any build.

`git push --follow-tags`
Branches don't carry tags; this does.

`gpg.format ssh + user.signingKey`
Sign with your SSH key: commit -S, tag -s.

Line endings B2

```
* text=auto
*.sh  text eol=lf
*.bat text eol=crlf
*.bin binary
```

`git add --renormalize .`
Fix files already committed with CRLF.

`git ls-files --eol`
What's stored, what's checked out.

Every command in the course is tested on every change. Lessons and exercises: consciouslycode.com/training · github.com/jhaws1982/git-power-user · Teams: the Git for teams workshop, consciouslycode.com/services